

Getränkebestellungen am Rattenfest

Studienarbeit
Herbstsemester 2026



Version: local
Datum: 27.09.2026

Team: Andrin Klarer
Jasmin Fässler
Betreuer: Frieder Loch

Departement Informatik
Ostschweizer Fachhochschule



Inhaltsverzeichnis

1	Problemstellung	6
1.1	Ausgangslage	6
1.1.1	Systemkontext und Domäne	6
1.2	Problem Domain	6
1.3	Ziel	7
2	Analyse	8
2.1	Marktanalyse	8
2.1.1	Sum-Up	8
2.2	Recherche User Experience während Rattenfest	8
2.2.1	wichtige Probleme	8
3	Vorarbeiten	9
3.1	Funktionsumfang des Prototyps	9
3.2	Erfahrungen aus dem Einsatz	9
3.3	Grenzen des Prototyps	10
3.4	Abgrenzung zu dieser Arbeit	10
3.5	Vorhandene Lösungswege «Case Study»	10
3.6	Weitere Technologien	10
3.7	Weitere Technologien	11
3.8	Ähnliche Lösungen	12
3.8.1	Webshop für Getränke	12
3.9	Fazit	12
3.10	Technologie Entscheidung	13
3.10.1	Kriterien des Frameworks	13
3.10.2	Vorauswahl	13
3.10.3	Informationen	15
3.11	React / Next.js mit Typescript	15
3.11.1	Updates	15
3.12	Angular	15
3.13	Vue / Vue.js	15
3.14	ASP.NET	16
3.15	Fazit	16
3.15.1	Risiken und Unsicherheiten	16
3.15.2	Technische, allgemeine Risikos	16
3.15.3	Spezielle Risikos	16
3.15.4	Akteure	17
3.15.5	User Stories und Functional Requirements	17
3.15.6	Non-Functional Requirements	21
3.15.7	Functional Requirements	23
3.15.8	Non-Functional Requirements	23
3.16	Anwendungsszenarien (detaillierter)	23
4	Methode	24
4.1	Design und Architektur	24
4.1.1	Prototyp / Skizze der Lösung	24
4.1.2	UI und Backend Konzept (Software-Architektur)	24
4.1.3	Zugangskonzept	24
4.2	Implementation	24
5	Ergebnisse	24

5.1	Evaluation	24
5.1.1	Functional Requirements	24
5.1.2	Non-Functional Requirements	24
6	Fazit und Ausblick	24
6.1	Retrospektive	24
6.2	Zukünftige Weiterentwicklung	24
7	Projektinformationen	25
7.1	Projektplan	25
7.1.1	Zeitplan	25
8	Hilfsmittelverzeichnis	26
9	Bilderverzeichnis	26
10	Tabellenverzeichnis	26
11	Bibliografie	27
12	Anhang A	28
12.1	APIs	28
12.2	Screenshots	28
12.3	Testreports	28
13	Anhang B	28
13.1	Meeting-Protokolle	28
13.2	Eigenständigkeitserklärung	28
13.3	Persönliche Bericht	28

Abstract

Management Summary

Vorgehen und Technologien

Ergebnisse

1 Problemstellung

1.1 Ausgangslage

Das Rattenfest ist ein jährlich stattfindendes Festival, das von Studierenden der OST organisiert wird und rund 3'000 Personen besucht. Die Getränke werden an mehreren Bars ausgeschenkt, die unabhängig vom Rattenfest betrieben werden. Damit nicht jede Bar ihre Versorgung selbst organisieren muss, betreibt das Rattenfest ein zentrales Getränkelager, aus dem die Bars während des Fests laufend beziehen.

Dieser Prozess wurde in den letzten Jahren versucht zunehmend zu digitalisieren, die bisherigen Lösungen blieben jedoch unzureichend.

1.1.1 Systemkontext und Domäne

Es gibt eine Website des Rattenfest `rattenfest.ch`, diese wird über Hostpoint gehostet. Das Ticket-system und die Bezahlung läuft extern über einen Anbieter. Die Kommunikation intern läuft über Microsoft Teams, E-Mails und Chatdienste wie WhatsApp. Dateien sind unter Microsoft Teams abgelegt, es gibt keinen Dateiserver. Es gibt bereits einen Prototyp für das Barsystem und als Hilfe für den OK im Vorverkauf wurde bereits eine kleine Website getestet. So sind die Schnittstellen vom Rattenfest sehr flexibel und die verschiedenen Services können unabhängig voneinander gewechselt werden.

1.2 Problem Domain

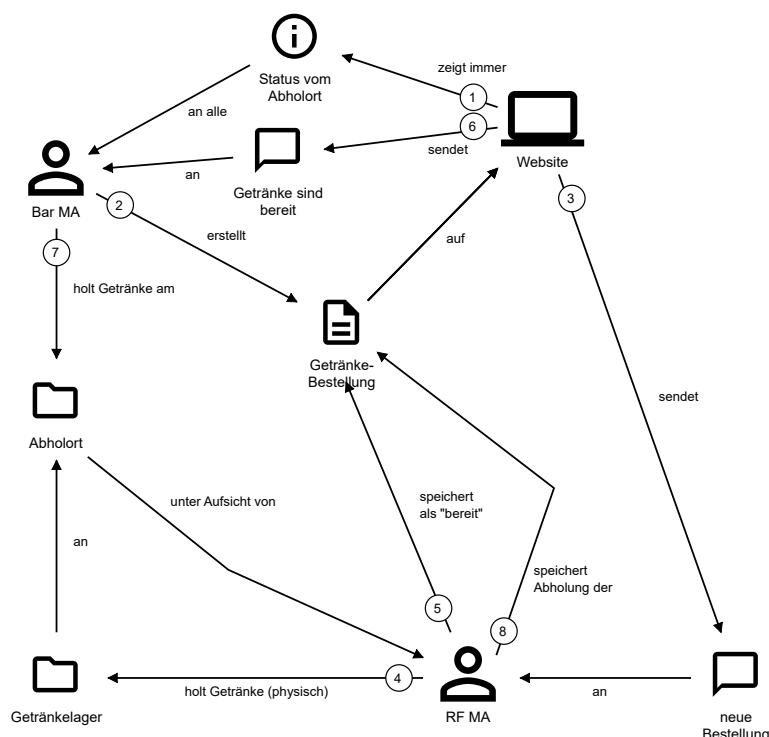


Abbildung 1: Problem Domain der Getränkebestellung

Umfang:

- Eine Getränke-Bestellung umfasst wenige bis zu mehreren hundert Artikel.
- Das Bereitstellen der Getränke (4) dauert je nach Grösse der Bestellung und Anzahl der eingehenden Bestellungen zwischen 2 und 10 Minuten.

- Gemäss den Erfahrungen aus dem letzten Jahr gehen zu Spitzenzeiten bis zu 10-15 Bestellungen pro 15 Minuten ein.
- Die Wege bei der Abholung (7) sind unterschiedlich lang. Bis man sich mit der Ware durch die Menschenmenge gekämpft hat, können mehrere Minuten vergehen.

Höchste Priorität bei der Bestellung: Solange ein Produkt an Lager ist, darf es an keiner Bar ausgehen. Das wird durch schnelle Kommunikation und Live-Updates sichergestellt.

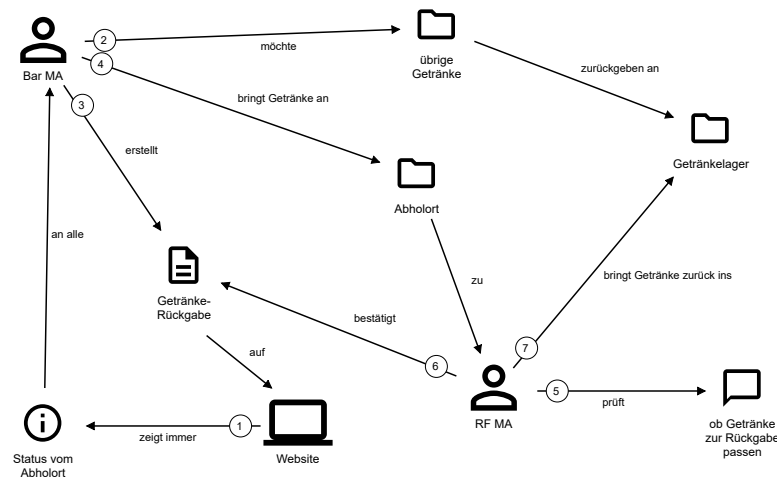


Abbildung 2: Problem Domain der Getränkerückgabe am Ende des Festes

Bei der Rückgabe ist das Fest vorbei und es darf deshalb auch länger dauern.

1.3 Ziel

Ziel dieser Arbeit ist eine Webapplikation, die den gesamten Lebenszyklus der Getränke abbildet, von der ersten Bedarfsschätzung bis zur Abrechnung nach dem Fest:

- **Vor dem Fest** erfassen die Bars ihre Bedarfsschätzung in der Applikation. Das OK prüft und korrigiert diese und leitet daraus die Bestellung beim Getränkehändler ab. Die gelieferte Ware bildet den Anfangsbestand des Lagers.
- **Während des Fests** bestellen die Bars laufend aus diesem Bestand. Das Lagerteam stellt die Bestellungen bereit und informiert die Bars über den Status.
- **Nach dem Fest** lässt sich für jede Bar eine detaillierte Abrechnung erstellen.

Neben der Funktionalität soll die Lösung auf das Rattenfest zugeschnitten und für den langfristigen Einsatz geeignet sein. Dabei sind drei Eigenschaften besonders wichtig:

- **Bedienbarkeit** unter den Bedingungen am Fest: dunkel, laut, unter Zeitdruck und mit Personal, dessen Aufmerksamkeit im Verlauf des Abends nachlässt.
- **Zuverlässigkeit** während des Fests. Sobald die Bars auf den manuellen Weg ausweichen müssen, verliert die Applikation ihren Zweck.
- **Verwendbarkeit und Wartbarkeit** durch ein jährlich wechselndes OK mit unterschiedlichen technischen Kenntnissen.

2 Analyse

2.1 Marktanalyse

2.1.1 Sum-Up

2.2 Recherche User Experience während Rattenfest

2.2.1 wichtige Probleme

3 Vorarbeiten

In den vergangenen Jahren wurde versucht, das Getränkemanagement am Rattenfest zunehmend zu digitalisieren. Für das Rattenfest 2026 wurde, unabhängig von dieser Arbeit und vor deren Beginn, erstmals ein Prototyp für die Getränkebestellungen während des Fests entwickelt und eingesetzt. Dieser Prototyp wird im Folgenden beschrieben und von den Ergebnissen dieser Arbeit abgegrenzt.

3.1 Funktionsumfang des Prototyps

Der Prototyp deckt ausschliesslich den Bestellprozess während des Fests ab:

- Die Bars bestellen über eine Webseite Getränke aus dem Lager.
- Das Lagerteam erhält die Bestellungen, bearbeitet diese und stellt sie bereit.
- Die Bars holen die bereitgestellten Getränke im Lager ab.
- Am Ende des Fests können Retouren erfasst werden.
- Nach dem Fest stehen die Daten zur Verfügung, um pro Bar eine Abrechnung zu erstellen.

Für die Benutzeroberfläche wurde die Komponentenbibliothek shadcn/ui [1] verwendet, die unter anderem einen Dark Mode mitbringt. Die Applikation übernahm dabei die Einstellung des Geräts und wurde dunkel dargestellt, sofern dieses auf den Nachtmodus geschaltet war.

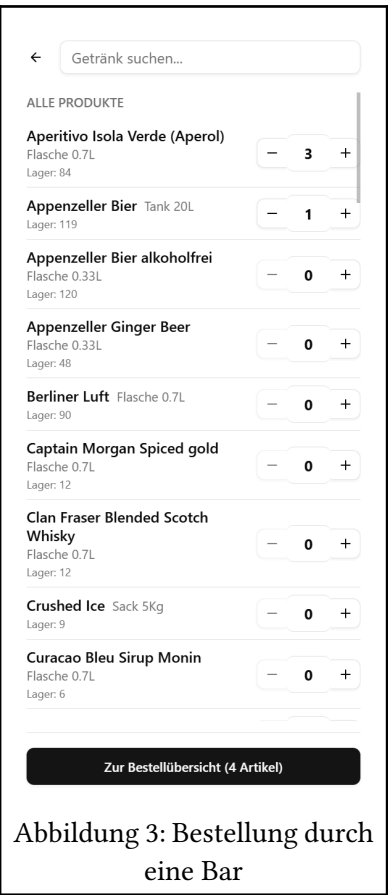


Abbildung 3: Bestellung durch eine Bar

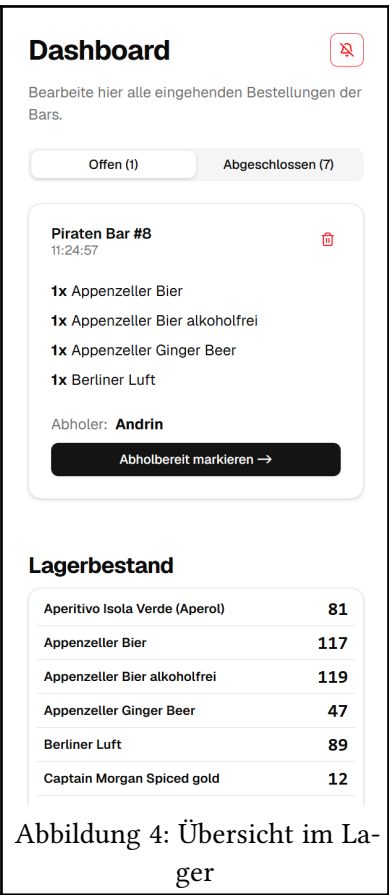


Abbildung 4: Übersicht im Lager

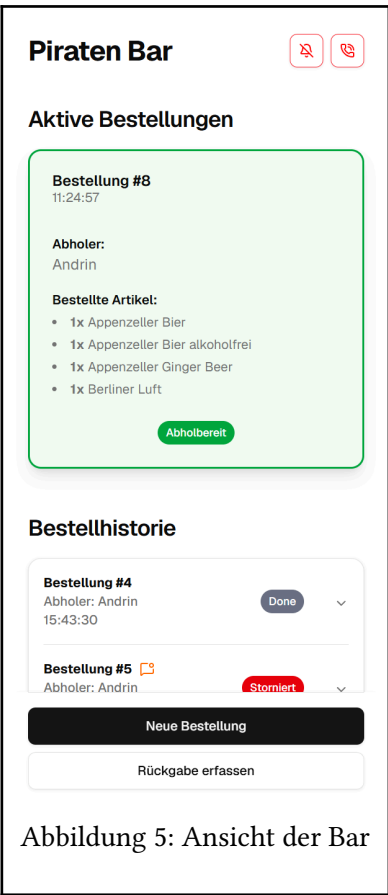


Abbildung 5: Ansicht der Bar

Tabelle 1: Ansichten des Prototyps

3.2 Erfahrungen aus dem Einsatz

Der Prototyp war während des gesamten Fests ohne nennenswerte Ausfälle im Einsatz. Die Rückmeldungen von Bar- und Lagerteam fielen überwiegend positiv aus. Der Einsatz unter realen Bedingungen

lieferte zudem konkrete Hinweise auf fehlende Funktionen, die ohne Prototyp kaum erkannt worden wären, beispielsweise:

- die Berücksichtigung von Pausen und Schichtwechseln,
- Angabe von Preisen und Übersicht der Ausgaben,
- Einfache Kontaktmöglichkeit zur Security.

Diese Erkenntnisse fließen als Anforderungen in diese Arbeit ein.

3.3 Grenzen des Prototyps

Trotz des erfolgreichen Einsatzes genügt der Prototyp den Zielen dieser Arbeit nicht:

- **Architektur und Qualität:** Der Prototyp wurde aufgrund mangelnder Planung als Einweglösung entwickelt. Es besteht keine Testabdeckung und keine saubere Architektur, was eine Weiterentwicklung und Wartung durch ein wechselndes OK erschwert.
- **Datenmodell:** Das Datenschema ist auf ein einzelnes Fest ausgelegt. Um die App für ein weiteres Fest zu verwenden, muss die DB gewiped werden und sämtliche Daten gehen verloren, womit ein Vergleich zwischen Jahrgängen nicht möglich ist.
- **Funktionsumfang:** Abgedeckt ist nur der Bestell- und Rücknahmeprozess. Bedarfsschätzung und Vorbestellung, automatischer Lagerbestand, Preisaufstellungen und fertige Abrechnungen fehlen vollständig.
- **Administration:** Viele Aufgaben wie der Import von Getränken, Bars, Benutzern und Telefonnummern erfolgen über das Dashboard der Datenbank oder mit Skripten. Ohne technische Kenntnisse ist die Applikation nicht betreibbar.

3.4 Abgrenzung zu dieser Arbeit

Der Prototyp dient dieser Arbeit als validierte Ausgangslage, nicht als Codebasis, die unverändert übernommen wird. Da der Umfang dieser Arbeit deutlich grösser ist, erlauben wir uns bewiesene Konzepte zu übernehmen.

- der Technologie-Stack, da er sich im Einsatz bewährt hat und für ein wechselndes OK einfach zu warten ist. (Vercel & Supabase)
- der grundlegende Ablauf des Bestellprozesses & Retouren
- das Zugangskonzept: Bars greifen über einen persönlichen Link ohne Passwort zu, Lagerteam und Administratoren melden sich mit Benutzername und Passwort an.

Neu im Rahmen dieser Arbeit entstehen insbesondere ein mehrjahresfähiges Datenmodell, eine testbare Architektur, die Administration über die Applikation selbst sowie die Prozesse vor und nach dem Fest. Zudem wird die Bedienbarkeit unter Festbedingungen, auf Basis der Recherche in und der Rückmeldungen aus dem Einsatz, untersucht und weiterentwickelt.

3.5 Vorhandene Lösungswege «Case Study»

Sum-up

3.6 Weitere Technologien

Neben Bestellsystemen haben wir stichprobenweise Technologien aus verwandten Bereichen betrachtet. Leitfrage war jeweils, ob sie einen Arbeitsschritt am Rattenfest (Bestellen, Bereitstellen, Abholen, Rückgabe) vereinfachen würden.

Automatische Bestandserfassung. Die «real-time event inventory application» von rapidstock ermittelt den Inhalt offener Flaschen über eine Bluetooth-Waage und plant das Inventar anhand der

erwarteten Gästezahl [2]. Ein verwandter Ansatz ist das Zählen per Bilderkennung. Dies wurde für die Schlussinventur des Rattenfests bereits getestet, lieferte aber zu ungenaue Ergebnisse. Auch Starbucks stellte ein vergleichbares System nach wenigen Monaten wieder ein [3]. Für das Rattenfest bringen beide Ansätze wenig: Das Lager gibt ganze Gebinde aus, deren Zählung heute kein Problem ist. Die Gästezahl ist kaum planbar, da rund 60-70 % der Tickets erst am Festtag verkauft werden.

Bezahlung bei der Bestellung. Zahlungslösungen wie SumUp mit kontaktloser Zahlung per NFC [4] oder TWINT mit QR-Code [5] würden es erlauben, Getränke direkt bei der Bestellung zu bezahlen. Für das Rattenfest ist das nicht sinnvoll, da die Bars ihren Getränkebezug nicht vorfinanzieren sollen. Die Einnahmen der Bars fließen zuerst an das Rattenfest, das nach dem Fest den Getränkebezug abzieht und die Differenz auszahlt. Die Bezahlung ist damit Teil der Abrechnung (FR-26) und nicht des Bestellprozesses, eine Zahlungsintegration wird nicht umgesetzt.

Rückmeldung an die Nutzenden. Übertragbar ist hingegen die Art der Rückmeldung: SumUp bestätigt eine Zahlung mit einem Signalton und vier LEDs [4]. Die Bestätigung wird wahrgenommen, ohne dass eine Nachricht gelesen werden muss. Unter Festbedingungen (dunkel, laut, Zeitdruck) ist das besonders wertvoll und fließt in NFR-03 ein. Da ein Ton im Lärm untergehen kann, sollte er mit gut sichtbaren Farben kombiniert werden.

Fazit. Die automatische Bestandserfassung löst ein Problem, das am Rattenfest nicht besteht, und rechtfertigt den Aufwand nicht. Auf eine Zahlungsintegration wird bewusst verzichtet, da die Bezahlung über die Abrechnung nach dem Fest erfolgt. Übernommen wird das Prinzip der Rückmeldung über mehrere Kanäle (Ton und Farbe), das in NFR-03 einfließt. Eine automatische Erfassung könnte sich lohnen, falls Bars ihre Getränke künftig selbst aus dem Kühlwagen holen (siehe Abschnitt 6.2).

3.7 Weitere Technologien

Wir haben stichprobenweise Lösungen verschiedener Softwareanbieter angeschaut, um herauszufinden, ob sie in unserem Fall einen Vorteil bringen würden.

Die «real-time event inventory application» von «rapidstock» bietet eine Waage, die das Gewicht einer offenen Flasche per Bluetooth an die Applikation sendet [2]. Übertragen auf unseren Fall müsste nur noch der Getränketyp angegeben werden, die Anzahl liesse sich dann über das Gewicht berechnen. Die Software bezieht ausserdem die erwartete Anzahl Gäste mit ein, um das Inventar besser planen zu können.

Beim Rattenfest ist das Zählen der Artikel aktuell kein Problem. Auch die Anzahl Gäste lässt sich im Voraus kaum abschätzen, da rund 60-70 % der Tickets erst am Festtag verkauft werden.

Sum-Up akzeptiert Bezahlungen mit NFC, damit können Bezahlungen in der Nähe von bis zu 10 cm kontaktlos getätigt werden. Ausserdem wird die Zahlung mit einem «Beep» und vier LEDs bestätigt. [4] Das spart «Klicks» auf dem Smartphone und das Lesen der Bestätigungsnachricht durch einfache Kommunikation.

Twint hingegen ist bekannt durch die Bezahlung per QR-Code, dadurch können viele Daten die mühsam einzutippen wären, schnell an ein fremdes Gerät übermittelt werden. [5]

Da wir bereits eine Verbindung aufgebaut haben und so einfach Daten senden können, benötigen wir NFC oder einen QR-Code nicht. Es würde die Website unnötig komplex und mehr fehleranfällig machen. Ob eine Bestellung oder ein Getränk mühsam zu finden ist, können wir mit User Tests herausfinden. Das Problem könnte dann auch mithilfe von einer Suche oder einem Filter minimiert werden.

Benachrichtigungen mit «Beep» und gut sichtbaren Farben zu zeigen kann das Erfassen der Informationen für die Nutzenden verbessern (NFR-03).

Um das Getränkelager am Ende vom Rattenfest einfacher zu erfassen, wurde bereits getestet die Getränke mit Bilderkennung zu zählen. Jedoch ist das Ergebnis nicht genügend ausgereift und auch Starbucks hatte diese Technologie nach 9 Monaten wieder abgeschafft. [3]

3.8 Ähnliche Lösungen

3.8.1 Webshop für Getränke

Ein Webshop der Getränke verkauft ermöglicht eine gute Benutzerfreundlichkeit durch farbige und auffallende Bilder. Ausserdem gibt es einen Filter für die Getränke, ebenfalls mit einem Bild. Einzelne Getränke und 6er Packs werden in dieser Anzahl auf dem Bild abgebildet. [6]

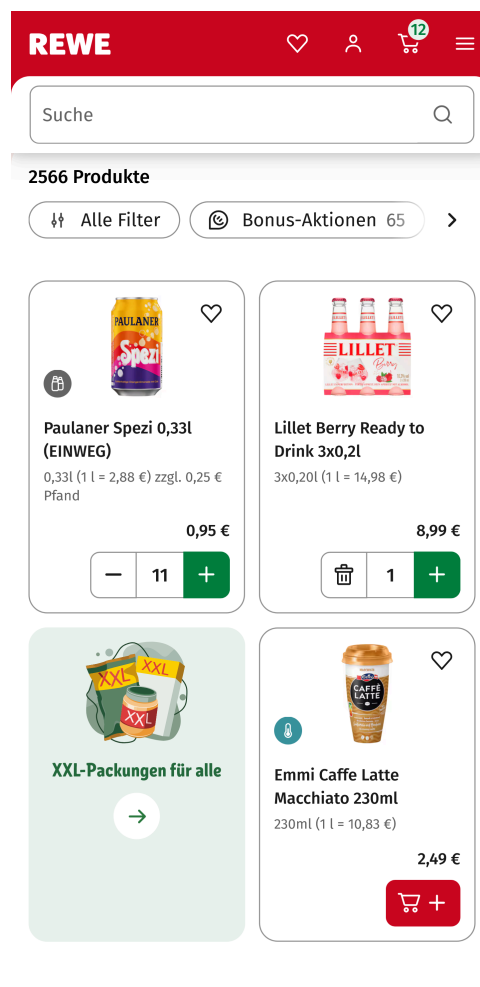


Abbildung 6: Beschreibung

3.9 Fazit

Es wurden Möglichkeiten gefunden, die Getränke schneller zu erfassen

3.10 Technologie Entscheidung

3.10.1 Kriterien des Frameworks

1. einfache Infrastruktur
 - 1a? zentrale Möglichkeit zum Testen und Updaten für Nicht-Informatiker möglich
 - 1b? langlebig/bewährt,
 - 1c? Unabhängigkeit vom Framework, erleichtert das Wechseln auf ein anderes Framework (Zusammenhang zu «einfach zu updaten»)
2. PWA mit Benachrichtigungen möglich
3. Geeignet für kleine/mittelgrosse Website
4. Einfacher Einstieg, einfach zu lernen

3.10.2 Vorauswahl

Die Performance ist in unserem Fall nicht abhängig vom Framework, sondern vom Hosting-Provider. Der Unterschied zwischen React und Angular beträgt nur Millisekunden [7]. Ausserdem verbessern die Frameworks ihre Performance laufend [7].

Wir suchen eine Lösung mit Client-Side Rendering, damit neuer Inhalt ohne ein Neuladen angezeigt wird.

Für das Hosting kennen wir bereits Vercel und Supabase für die Datenbankbindung, dieses eignet sich für React, Vue und Angular.

Es wurden die 4 momentan grössten Frontend Web Frameworks analysiert und nach den Kriterien verglichen [8]. Diese Frameworks besitzen eine gute Dokumentation und viele Nutzer.

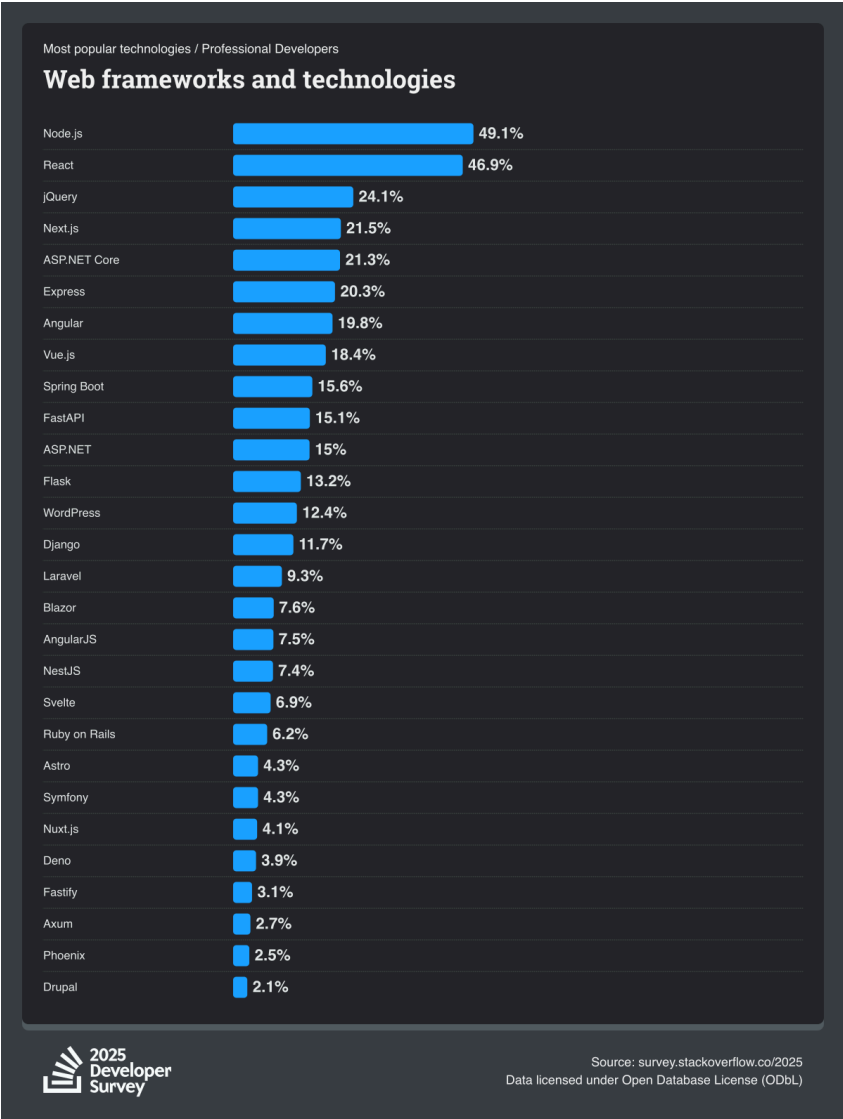


Abbildung 7: Stack Overflow Umfrage 2025

<https://2025.stateofjs.com/en-US/libraries/>

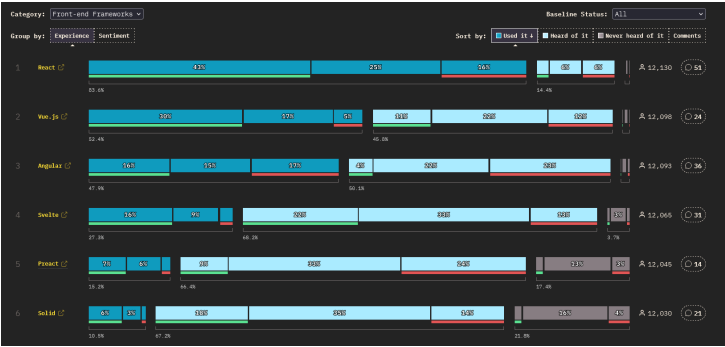


Abbildung 8: Beschreibung

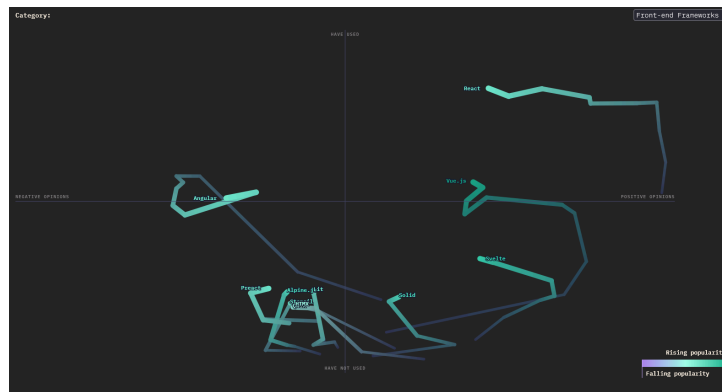


Abbildung 9: Beschreibung

3.10.3 Informationen

3.11 React / Next.js mit Typescript

1. Aufwändigere erste Einrichtung, Testing und Builder muss manuell eingerichtet werden.

Wir können eine Vorlage wie den T3Stack verwenden, durch die vielen Abhängigkeiten steigt aber die Komplexität und Aufwand beim Updaten. Ausserdem wird dann Next.JS verwendet, obwohl wir kein Server-Side Rendering benötigen. [9]

2. PWA ist möglich mit create-react-app (deprecated), Next.JS (übertrieben) <https://www.overcode.tech/blog/react-pwa> oder mit Vite + PWA von Vite (dokumentiert) <https://vite-pwa-org.netlify.app/guide/>
3. React eignet sich gut für diese Projektgrösse. [10]
4. Da wir bereits React kennen ist für uns der Einstieg einfach. Für React muss man JSX verstehen.

3.11.1 Updates

Info: Major Release enthält neue Features und möglicherweise das entfernen von veralteten Funktionen. Es ist möglicherweise nötig, den Code upzudaten, neue Tests zu schreiben und neue APIs zu lernen. Bugfixes und Sicherheitsupdates sind auch im LTS (Long-Term Support) nötig, aber der Code muss nicht verändert werden.

React wurde 2013 veröffentlicht. [11]. Es gibt keine Information in der Dokumentation, wie häufig es einen neuen «Major Release» gibt. Aber React veröffentlichte seit 2020 alle zwei Jahre einen neuen «Major Release» mit «Breaking Changes» [12].

3.12 Angular

1. Einfache und schnelle Einrichtung ohne die Datenbankanbindung. [13]
 - Der LTS (Long-Term Support) beträgt 12 Monate, dass bedeutet Angular veröffentlicht jedes Jahr einen neuen Major Release. [14]
2. PWA integriert und gut dokumentiert von Angular [15]
3. Angular gemacht für grosse Unternehmens-Systeme. [10]
4. Angular hat eine hohe Lernkurve durch die starke und Meinungsstarke (opinionated) Architektur. [16] Das Vorwissen von React und Typescript macht den Einstieg leichter.

3.13 Vue / Vue.js

[17]

1. Einfache Einrichtung und die Tools (JSX Support, Vitest, Router..) können direkt ausgewählt werden.
2. PWA möglich <https://cli.vuejs.org/core-plugins/pwa.html#configuration> <https://www.vuemastery.com/blog/getting-started-with-pwas-and-vue3/#starting-a-pwa-from-scratch>
3. Gemacht für kleine Projekte aber lässt sich auch gut skalieren.
4. Einfach zu lernen, da alles im Framework integriert ist und der Syntax einfacher ist als React. [18]

3.14 ASP.NET

Blazor lässt C# Code im Browser in einer WebAssembly .NET Laufzeitumgebung (Runtime) laufen [19]. Jedoch muss es zuerst die WebAssembly Laufzeitumgebung herunterladen, was die Ladezeit im Vergleich zu JS-basierten Webapps verlangsamt [20].

1. Das Setup ist sehr ausführlich erklärt, es wird keine Shell benötigt wie in den anderen Frameworks. [21]
2. Die PWA Einrichtung ist ausführlich dokumentiert. Für die Push-Benachrichtigungen wird ein Backend Server benötigt. [22]
3. Geeignet für grosse Websites und Unternehmen die bereits .NET nutzen.
4. Es wird in C# programmiert und es werden Komponente genutzt ähnlich zu React. [21] Für das HTML wird im Gegensatz zu React, welches JSX nutzt, Razor verwendet [23]. So muss ein zusätzlicher Syntax gelernt werden.

3.15 Fazit

3.15.1 Risiken und Unsicherheiten

3.15.2 Technische, allgemeine Risikos

Sicherheit Code .. Ausfall

3.15.3 Spezielle Risikos

Risiko eine Funktion zu vergessen Massnahme: Zielgruppe befragen, Im Voraus mit Prototyp und im Nachhinein User Tests machen

Risiko, dass es auf einer Plattform/Betriebssystem/Bildschirmgrösse nicht funktioniert wie gedacht
Massnahme: tests

Risiko, dass sich die Benutzung durch die spezielle Umgebung verschlechtert. Spezielle Umgebung: während Rattenfest

- ausserhalb von der Website: Tipps für Bars, wie sie das Gerät mit der Website einrichten können, damit es möglichst integriert und nicht lästig ist (Gerät, Halterung, Rolle zuweisen ?)
- Bedienung von Website (User Experience) eng, dunkel, laut, Bedienung eingeschränkt

Massnahme: Recherche, stärkerer Fokus auf Accessibility

Risiko, die Internetverbindung ist zu langsam, da es sehr viele Menschen hat
Massnahme: Tests
> ist das Problem auch mit anderen Diensten? Protokolle analysieren, was lässt sich verbessern?
Nutzwertanalyse für teureres, lokales Hosting oder günstiges Hosting mit weniger Setupaufwand?
Möglichkeiten für Hosting am Rattenfest mit einfachem Setup? Bluetooth? Etwas wie Walkie Talkie?
Seiteninhalt klein halten und auf Smartphones cachen. Inhalt anzeigen + funktionen, auch wenn es noch nicht alles geladen hat.

Risiko, dass die nachfolgenden RF-OK Personen die Website nicht bedienen können und es nach z. B. zwei OK-Wechsel nicht mehr benutzt wird. Massnahme: kurze, einfach verständliche, einfach auffindbare und erreichbare Anleitungen erstellen. Kontakt vermerken für Fragen. Wissen gut weitergeben.

3.15.4 Akteure

- **Bar-Team:** bestellt Getränke für seine Bar und holt sie ab.
- **Lagerteam:** nimmt Bestellungen entgegen, stellt sie bereit und übergibt sie.
- **Getränkechef (RF-OK):** verwaltet Festaussage, Sortiment, Bars, Rücknahmen und Abrechnung.

3.15.4.1 Glossar

Die Festaussage bezeichnet die Speicherung der Daten von je einer Rattenfest-Durchführung.

- Das Sortiment bezeichnet das ursprüngliche Lagersortiment, gemäss der Vorbestellung vom Rattenfest. Es beinhaltet die Getränke mit Anzahl.
- Die Getränke beinhalten Informationen: Name, Stückzahl (manche Getränke können nur in 6er Päckchen bestellt werden) und Preis.
- RF-OK = Rattenfest Organisationskomitee Mitglied
- Schrumpfpackungen = ..
- Getränk / Artikel (vereinheitlichen?)

3.15.5 User Stories und Functional Requirements

3.15.5.1 Vorbereitung

US-01: Als RF-OK möchte ich eine neue Festaussage anlegen, damit die Daten jedes Jahrgangs getrennt bleiben und verglichen werden können.

- **FR-01:** Das System muss erlauben, eine Festaussage anzulegen und als aktiv zu setzen. Sortiment, Bestände, Bestellungen und Rückgaben sind jeweils einer Festaussage zugeordnet.
- **FR-02:** Das System muss erlauben, Sortiment und Bars einer früheren Festaussage als Ausgangslage zu übernehmen.

US-02: Als RF-OK möchte ich die Bars erfassen und ihnen Zugänge geben, damit Bestellungen eindeutig zugeordnet sind.

- **FR-03:** Das System muss erlauben, Bars mit Bezeichnung und Kontaktangabe zu erfassen, zu ändern und zu deaktivieren.
- **FR-04:** Das System muss pro Bar einen Zugang als teilbaren Link erzeugen und erlauben, diesen neu zu erzeugen, womit der bisherige ungültig wird.

US-03: Als RF-OK möchte ich das Getränkesortiment unseres Lieferanten den Bars zur Verfügung stellen.

- **FR-05:** Das System muss erlauben, Getränke mit Bezeichnung, Gebindegrösse und Preis zu erfassen.
- **FR-06:** Das System muss erlauben, ein Sortiment aus einer Datei zu importieren.

US-04: Als Bar-Team möchte ich meinen Bedarf vor dem Fest schätzen, damit die benötigte Ware eingekauft wird.

- **FR-07:** Das System muss erlauben, pro Bar eine Bedarfsschätzung über das Sortiment zu erfassen und bis zu einem Stichtag zu ändern.

US-05: Als RF-OK möchte ich die Schätzungen prüfen und zusammenfassen, damit ich daraus die Bestellung beim Getränkehändler ableiten kann.

- **FR-08:** Das System muss die Schätzungen aller Bars darstellen und einzeln korrigierbar machen.

US-06: Als RF-OK möchte ich die gelieferte Ware einlagern, damit der Lagerbestand von Beginn an stimmt.

- **FR-09:** Das System muss den Anfangsbestand aus der Bestellung beim Händler übernehmen.

3.15.5.2 Bestellung während des Fests

US-07: Als Bar-Team möchte ich online Getränke bestellen.

- **FR-10:** Das System muss erlauben, eine Bestellung über mehrere Artikel mit Mengenangabe abzuschicken.
- **FR-11:** Das System darf nur Mengeneingaben erlauben die den Gebindegrößen entsprechen.
- **FR-12:** Das System muss verhindern, dass mehr bestellt wird, als im Lager verfügbar ist.
- **FR-13:** Das System muss einen Hinweis anzeigen, wenn eine Bestellung einen Artikel enthält, der sich von der Reservation der Bar unterscheidet.

US-08: Als Bar-Team möchte ich eine abgeschickte Bestellung stornieren können.

- **FR-14:** Das System muss erlauben, eine Bestellung zu stornieren, solange das Lagerteam noch nicht bearbeitet hat.

US-09: Als Bar-Team möchte ich eine frühere Bestellung wiederholen.

- **FR-15:** Das System muss die Bestellhistorie der eigenen Bar anzeigen und erlauben, einen früheren Eintrag als Vorlage zu übernehmen und vor dem Absenden anzupassen.

3.15.5.3 Bearbeitung und Abholung

US-10: Als Lagerteam möchte ich eingehende Bestellungen sofort sehen, damit ich ohne Verzögerung mit dem Bereitstellen beginnen kann.

- **FR-16:** Das System muss Bestellungen und deren Statusänderungen bei Bar und Lagerteam ohne manuelles Neuladen aktualisieren.
- **FR-17:** Das System muss offene Bestellungen in der Reihenfolge ihres Eingangs darstellen.

US-11: Als Bar-Team möchte ich zeitnah Bescheid erhalten, sobald meine Bestellung bereitsteht.

- **FR-18:** Das System muss erlauben, eine Bestellung als abholbereit zu markieren.
- **FR-19:** Das System muss erlauben, die Abholung zu bestätigen, womit die Bestellung abgeschlossen wird.
- **FR-20:** Das System muss eine Benachrichtigung an das Bar-Team senden, wenn die Bestellung abholbereit ist.

US-12: Als Lagerteam möchte ich Abweichungen festhalten, damit sie bei der Abrechnung nachvollziehbar sind.

- **FR-21:** Das System muss erlauben, eine Bestellung mit Begründung zu stornieren oder als nicht abgeholt zu kennzeichnen.

US-13: Als Bar-Team möchte ich bei technischen Problemen oder Zwischenfällen jemanden erreichen.

- **FR-22:** Das System muss eine Kontaktmöglichkeit zum Lagerteam und Sicherheitsverantwortlichen anzeigen.

US-14: Als Lagerteam möchte ich eine Pause einlegen, damit die Bars wissen, dass Bestellungen vorübergehend nicht bearbeitet werden.

- **FR-23:** Das System muss erlauben, das Lager als vorübergehend nicht besetzt zu markieren, was den Bars angezeigt wird.

3.15.5.4 Rücknahme und Abrechnung

US-15: Als Bar-Team möchte ich nicht verkaufte Ware zurückgeben, damit sie mir nicht verrechnet wird.

- **FR-24:** Das System muss erlauben, eine Rückgabe analog zu einer Bestellung zu erfassen, begrenzt auf die von der Bar bezogenen Artikel und Mengen.
- **FR-25:** Das System muss verlangen, dass das RF-OK eine Rückgabe bestätigt, bevor sie dem Lagerbestand gutgeschrieben wird.

US-16: Als RF-OK möchte ich nach dem Fest einfach und schnell pro Bar abrechnen.

- **FR-26:** Das System muss pro Bar eine Aufstellung aller bezogenen und zurückgegebenen Artikel mit Betrag erzeugen.
- **FR-27:** Das System muss Abrechnung und Rohdaten der Bezüge als Datei exportieren.

US-17: Als RF-OK möchte ich den Verbrauch mehrerer Feste vergleichen, damit die Schätzung im Folgejahr besser wird.

- **FR-28:** Das System muss die Daten vergangener Feste erhalten und pro Artikel vergleichbar darstellen.

3.15.5.5 Klassifikation der Functional Requirements

Die Functional Requirements sind nach der MoSCoW-Methode in drei Kategorien eingeteilt.

Kategorie	Bedeutung
Muss	Ohne diese Anforderungen erfüllt das System seinen Zweck nicht.
Soll	Wichtig für Bedienbarkeit, Effizienz oder Zuverlässigkeit. Das System funktioniert ohne sie, aber mit geringerer Qualität.
Kann	Wünschenswerte Ergänzung, die entfällt, wenn die Zeit nicht reicht.

Tabelle 2: Kategorien der MoSCoW-Klassifikation

ID	Kurzbezeichnung	Priorität
FR-01	Festausgabe anlegen und aktiv setzen	Muss
FR-02	Stammdaten aus Vorjahr übernehmen	Soll
FR-03	Bars verwalten	Muss
FR-04	Bar-Zugang erzeugen und erneuern	Muss
FR-05	Sortiment verwalten	Muss
FR-06	Sortiment importieren	Kann
FR-07	Bedarfsschätzung erfassen	Muss
FR-08	Schätzungen aggregieren und korrigieren	Muss
FR-09	Anfangsbestand erfassen	Muss
FR-10	Bestellung absenden	Muss
FR-11	Schrittweite für Mengeneingabe	Soll
FR-12	Bestand als Obergrenze	Muss
FR-13	Hinweis bei auffälliger Bestellung	Kann
FR-14	Bestellung stornieren (Bar)	Muss
FR-15	Historie anzeigen und wiederverwenden	Soll
FR-16	Aktualisierung ohne Neuladen	Muss
FR-17	Offene Bestellungen nach Eingang	Muss
FR-18	Als abholbereit markieren	Muss
FR-19	Abholung bestätigen	Muss
FR-20	Benachrichtigung ausserhalb der App	Kann
FR-21	Bestellung kennzeichnen (Lager)	Soll
FR-22	Kontaktmöglichkeit	Soll
FR-23	Lager als nicht besetzt markieren	Kann
FR-24	Rückgabe erfassen	Soll
FR-25	Rückgabe bestätigen	Soll
FR-26	Abrechnung pro Bar	Muss
FR-27	Datenexport	Soll
FR-28	Vergleich über Festausgaben	Kann

Tabelle 3: Klassifikation der Functional Requirements

3.15.6 Non-Functional Requirements

Die folgenden Anforderungen gelten für das System als Ganzes. Zu jeder ist angegeben, auf welche Functional Requirements sie sich bezieht und woran ihre Erfüllung gemessen wird. Da das Rattenfest nach Abgabe dieser Arbeit stattfindet, sind alle Anforderungen ohne das Fest überprüfbar. Im Vordergrund stehen die Zuverlässigkeit während des Fests und die Betreibbarkeit über mehrere Jahre. Skalierbarkeit, Redundanz und Barrierefreiheit sind bewusst niedrig angesetzt, da das System nicht öffentlich zugänglich ist und von weniger als 20 gleichzeitig verbundenen Personen genutzt wird.

NFR-01 (Konsistenz der Bestandsführung) betrifft FR-10, FR-12, FR-24

Eine Bestellung wird vollständig oder gar nicht verbucht, und auch bei wiederholtem Absenden genau einmal. Bei gleichzeitigen Bestellungen mehrerer Bars auf denselben Artikel darf die Summe der bestätigten Bezüge den verfügbaren Bestand nicht überschreiten und der Bestand nicht negativ werden.

Messung: Manueller Test, der zwei Bestellungen auf einen Artikel mit Bestand 1 gleichzeitig absetzt: genau eine wird bestätigt, die andere abgelehnt, keine verbleibt in einem Zwischenzustand. Zusätzlich wird dieselbe Bestellung mehrfach abgesendet und darf nur einmal im Bestand erscheinen.

NFR-02 (Latenz der Statusaktualisierung) betrifft FR-16, FR-18

Eine Statusänderung ist beim jeweils anderen Akteur innerhalb von zwei Sekunden sichtbar (95. Perzentil), bei 20 gleichzeitig verbundenen Bar-Geräten.

Messung: Lasttest mit simulierten Clients, gemessen wird die Zeit zwischen dem Auslösen der Statusänderung und ihrem Eintreffen beim Empfänger.

NFR-03 (Bedienbarkeit unter Festbedingungen) betrifft FR-10, FR-15

[

- Benachrichtigungen mit «Beep» und Farben nutzen

]

Messung:

NFR-04 (Geräte- und Bildschirmunterstützung)

Alle Funktionen sind auf aktuellem Chrome (Android), Safari (iOS) und einem Desktop-Browser bei Bildschirmbreiten von 320 bis 1920 Pixeln vollständig nutzbar.

Messung: Manuelle Testmatrix über die definierten Kombinationen aus Browser, Betriebssystem und Bildschirmbreite.

NFR-05 (Nachvollziehbarkeit der Abrechnung) betrifft FR-21, FR-26, FR-27

Jede Bestandsänderung, also Bezug, Rückgabe und Korrektur, ist dauerhaft mit Zeitpunkt, Bar und auslösendem Benutzer gespeichert, sodass jeder Betrag der Abrechnung auf einzelne Vorgänge zurückführbar ist.

Messung: Die Abrechnung einer Testbar wird stichprobenweise auf die zugrunde liegenden Einzelsvorgänge zurückgerechnet.

NFR-06 (Betreibbarkeit durch das OK) betrifft FR-01 bis FR-05, FR-09

Alle wiederkehrenden Betriebsaufgaben, also das Anlegen einer Festausgabe, die Verwaltung von Bars und Sortiment, Bestandskorrekturen und die Abrechnung, sind über die Oberfläche ausführbar und erfordern keinen direkten Datenbankzugriff. Eine nicht am Projekt beteiligte Person richtet anhand der Anleitung eine neue Festausgabe in unter 30 Minuten ein.

Messung: Nachweis der Oberflächenabdeckung anhand der Liste der Betriebsaufgaben, Durchführung der Einrichtung durch eine projektfremde Person.

NFR-07 (Zugriffsschutz der Bar-Zugänge) betrifft FR-04

Ein Bar-Zugang ist nicht erratbar (mindestens 122 Bit Entropie) und gewährt ausschliesslich Zugriff auf die Daten der eigenen Bar. Administrative Funktionen sind über diesen Zugang nicht erreichbar.

Messung: Negativtests gegen fremde Bar- und Administrationsendpunkte mit einem gültigen Bar-Zugang.

3.15.7 Functional Requirements

3.15.7.1 RF Admin

- Admin Dashboard

Immer:

- Analyse/Statistics: was wäre wichtig? Export als Excel/CSV der Daten (wie? wo speichern?)
Vergleiche zwischen verschiedenen Jahren. Bessere Schätzungen

3.15.7.2 Bar

- Login mit erhaltenen Daten
- Getränke bestellen (Bar)
- Undo/Redo (siehe oben)
- History > alte Bestellungen betrachten, prüfen, wiederholen
- Abholung > Identität verifizieren nötig? Abholung bestätigen
- Kontaktfunktion (allgemeine Probleme/Fragen melden, Notfallmeldungen während Fest, wer von RF behandelt Meldungen?)

3.15.8 Non-Functional Requirements

3.16 Anwendungsszenarien (detaillierter)

1. Vorbereitung & Inventar
 - Ersteinlagerung: Die gesamte eingekaufte Ware wird digital erfasst und bildet das zentrale Live-Inventar.
2. Bestellprozess für Bars
 - Zugang: Jede Bar erhält einen eigenen Zugangslink (z.B. app.com/UUID).
 - Bestelllogik:
 - Bars bestellen flexibel nach Bedarf.
 - Nachbestellungen über die ursprüngliche Schätzung hinaus sind möglich, solange der Artikel im zentralen Inventar verfügbar ist.
 - Optional: Ein automatisches Warnsystem meldet den Organisatoren kritisch tiefe Lagerbestände, erlaubt nur noch eine bestimmte Anzahl pro Bestellung (Falls eigenes Limit bereits erreicht ist).
3. Abwicklung & Abholung
 1. Bestellung: Die Bar sendet eine Bestellung über die App ab.
 - Bestellung für wenige verschiedene Getränke
 - Bestellung für ein Getränk in grossen Mengen
 - eine Bestellung von vorher anpassen und nochmals bestellen
 - a. Bar kann die Bestellung stornieren, anpassen und neu senden.
 2. Benachrichtigung: Das RF-Team wird sofort digital informiert.
 3. Bereitstellung: Das Team stellt die Ware zusammen und markiert die Bestellung in der App als «Abholbereit». a. RF-Team sieht Problem und kann Bestellung stornieren/markieren.
 4. Abholung: Die Bar sieht das Status-Update in Echtzeit und holt die Ware ab, was unnötige Wege vermeidet. a. RF-Team kann Bestellung stornieren/markieren wenn zu lange nicht abgeholt.
0. Bar/RF kann bei technischen Problemen eine Person vom RF mit einer Notfallmeldung erreichen.

4. Prozess für Getränke Rücknahmen

1. Bars können wie bei einer Bestellung eine Rückgabe erfassen und bringen die Ware zum Lager.
 2. Admin-Funktion: Nur ein Admin kann Rücknahmen bearbeiten... (Wichtig bei Rückgabe von Schrumpfpackungen, da diese nur als Einheit angenommen werden)
5. Erweitert Analytics, Umsatz pro Bar, pro Getränk etc. Export von Bezugslisten, Bestellhistorie, Rechnung etc. Initiale Bestellung, Getränkewünsche ebenfalls über die App

Weitere Features, z.B. «3.1 Bestellung» «die Bestellung für später vorbereiten und noch nicht senden, RF kann diese bereits sehen und sich besser vorbereiten»

- Konkurrenzanalyse durchführen, schauen was die gut machen und wir übernehmen können, ergänzen.
- User Befragung, wäre es sinnvoll oder unnötig
- Abwägen wie gross der Aufwand für den Nutzen ist

4 Methode

4.1 Design und Architektur

4.1.1 Prototyp / Skizze der Lösung

4.1.2 UI und Backend Konzept (Software-Architektur)

4.1.3 Zugangskonzept

4.2 Implementation

5 Ergebnisse

5.1 Evaluation

5.1.1 Functional Requirements

5.1.2 Non-Functional Requirements

6 Fazit und Ausblick

6.1 Retrospektive

6.2 Zukünftige Weiterentwicklung

7 Projektinformationen

7.1 Projektplan

Der Aufwand beträgt 2 Arbeitstage pro Woche für ca. 17h/W , insgesamt 240 Stunden Aufwand.

Hier ist der Zeitplan mit den wichtigsten Meilensteinen, alle weiteren Planungen zur Projektarbeit befinden sich in den Github Issues vom Projekt.

7.1.0.1 M1: Recherche fertig

Die Recherche (Literatur und Befragung von Usern) wurde ausgewertet und es wurden mindestens drei Beschlüsse daraus gezogen, die sich in den FR oder NFR zeigen.

7.1.0.2 M2: Umsetzung, Programmierung abgeschlossen

Die Umsetzung der FR und NFR ist abgeschlossen und die Website ist im Internet erreichbar.

7.1.0.3 M3: Dokumentation bereit zum Korrekturlesen

Die Dokumentation beinhaltet alle nötigen Informationen gemäss dem Leitfaden und ist bereit zum Korrekturlesen.

7.1.1 Zeitplan

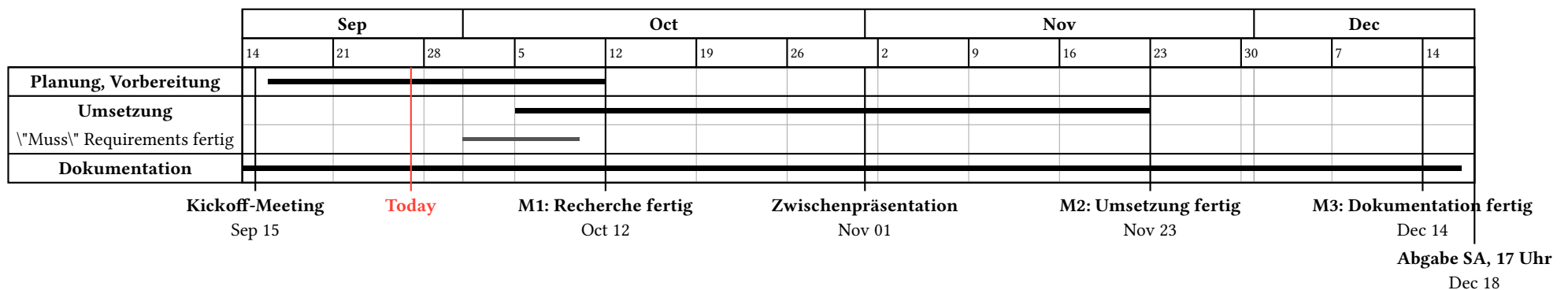


Abbildung 10: Langfristiger Zeitplan

8 Hilfsmittelverzeichnis

Die folgende Übersicht dokumentiert die im Projekt verwendeten Hilfsmittel nach Aufgabenbereich. Die konkreten Tools werden im Verlauf des Projekts ergänzt.

Aufgabenbereich	Tools
Literatur-Recherche und Verwaltung	Google, DuckDuckGo, scholar.google.com
Datenanalyse und Visualisierung	
Ideengenerierung	Claude, Gemini, Typst
Übersetzung	
Prototyping	Figma
Coding	Claude Code
Texterstellung, Textoptimierung, Rechtschreib- und Grammatikprüfung	Claude, Hunspell
Zusammenarbeit und Projektmanagement	Teams, GitHub, Outlook, Google Meet, Typst, WhatsApp
DevOps	GitHub
Notizen	Obsidian, Papier

Tabelle 4: Hilfsmittelverzeichnis

9 Bilderverzeichnis

Abbildung 1	Problem Domain der Getränkebestellung	6
Abbildung 2	Problem Domain der Getränkerückgabe am Ende des Festes	7
Abbildung 3	Bestellung durch eine Bar	9
Abbildung 4	Übersicht im Lager	9
Abbildung 5	Ansicht der Bar	9
Abbildung 6	Beschreibung	12
Abbildung 7	Stack Overflow Umfrage 2025	14
Abbildung 8	Beschreibung	14
Abbildung 9	Beschreibung	15
Abbildung 10	Langfristiger Zeitplan	25

10 Tabellenverzeichnis

Tabelle 1	Ansichten des Prototyps	9
Tabelle 2	Kategorien der MoSCoW-Klassifikation	19
Tabelle 3	Klassifikation der Functional Requirements	20
Tabelle 4	Hilfsmittelverzeichnis	26

11 Bibliografie

- [1] shadcn, «shadcn/ui». Zugegriffen: 27. September 2026. [Online]. Verfügbar unter: <https://ui.shadcn.com/>
- [2] «Event inventory application». [Online]. Verfügbar unter: <https://rapidstock.com/event-inventory-application>
- [3] S. Rogelberg, «Starbucks quietly retired its AI agent just months after deployment after it miscounted coffee shop inventories and slowed down baristas», *Fortune*, Mai 2026, [Online]. Verfügbar unter: <https://fortune.com/2026/05/28/starbucks-quietly-retired-ai-inventory-agent-barista-complaints-hallucinations>
- [4] «Accepting Contactless (NFC) Payments - SumUp - Support Center». [Online]. Verfügbar unter: <https://help.sumup.com/en-US/articles/722OhVEWYtfmy7X4959aJi-accepting-contactless-nfc-payments>
- [5] «QR-Code-Sticker - TWINT». [Online]. Verfügbar unter: <https://www.twint.ch/geschaefstkunden/produkte/twint-vor-ort/qr-code-sticker>
- [6] «Getränk online bestellen und liefern lassen - REWE.de». [Online]. Verfügbar unter: <https://www.rewe.de/shop/c/getraenke-genussmittel/?source=mc>
- [7] «Angular vs React: 1 Clear Winner in 2026 [7 Tests]». [Online]. Verfügbar unter: <https://techinsider.org/angular-vs-react-2026>
- [8] «Technology 2025 Stack Overflow Developer Survey». [Online]. Verfügbar unter: <https://survey.stackoverflow.co/2025/technology#most-popular-technologies-webframework-webframe-prof>
- [9] «Create T3 App». [Online]. Verfügbar unter: <https://create.t3.gg/>
- [10] «How to Choose the Right Front-End Framework for Your Project». [Online]. Verfügbar unter: <https://devtechinsights.com/choose-right-frontend-framework>
- [11] «React Versions – React». [Online]. Verfügbar unter: <https://react.dev/versions>
- [12] «Versioning Policy – React». [Online]. Verfügbar unter: <https://react.dev/community/versioning-policy>
- [13] A. Team, «Installation Angular». [Online]. Verfügbar unter: <https://angular.dev/installation>
- [14] A. Team, «Versioning and releases Angular». [Online]. Verfügbar unter: <https://angular.dev/reference/releases>
- [15] A. Team, «Push notifications Angular». [Online]. Verfügbar unter: <https://angular.dev/ecosystem/service-workers/push-notifications>
- [16] S. Khathi Chhetri, «Comparative Study of Front-end Frameworks : React and Angular». [Online]. Verfügbar unter: <https://www.theseus.fi/items/264baf5d-1b0e-4489-9fcb-93bd65d16223>
- [17] «Vue.js». [Online]. Verfügbar unter: <https://vuejs.org/>
- [18] G. Sourabh, «Vue vs React: Which is the Best Frontend Framework in 2025? BrowserStack», *BrowserStack*, Aug. 2025, [Online]. Verfügbar unter: <https://www.browserstack.com/guide/react-vs-vuejs>

- [19] guardrex, «ASP.NET Core Blazor». [Online]. Verfügbar unter: <https://learn.microsoft.com/en-us/aspnet/core/blazor/?view=aspnetcore-10.0#build-a-full-stack-web-app-with-blazor>
- [20] «Blazor vs React Performance - Which Is Faster in 2025?». [Online]. Verfügbar unter: <https://brisktechsol.com/blazor-vs-react>
- [21] «Blazor tutorial - Build your first web app \vert .NET». [Online]. Verfügbar unter: <https://dotnet.microsoft.com/en-us/learn/aspnet/blazor-tutorial/intro>
- [22] guardrex, «ASP.NET Core Blazor Progressive Web Application (PWA)». [Online]. Verfügbar unter: <https://learn.microsoft.com/en-us/aspnet/core/blazor/progressive-web-app/?view=aspnetcore-10.0&tabs=visual-studio>
- [23] danroth27, «Build reusable UI components with Blazor - .NET». [Online]. Verfügbar unter: <https://learn.microsoft.com/en-us/dotnet/architecture/blazor-for-web-forms-developers/components>

12 Anhang A

12.1 APIs

12.2 Screenshots

12.3 Testreports

13 Anhang B

13.1 Meeting-Protokolle

Meetingnotizen werden separat erstellt, hier werden nur wichtige Entscheidungen notiert.

- Wöchentliches Meeting am Dienstag mit Betreuer

13.2 Eigenständigkeitserklärung

13.3 Persönliche Bericht

Zusammenfassung unserer persönlichen Erfahrungen, welche wir aus dem Projekt mitnehmen.

